From this point on, so long as the geom_label.ko kernel module is loaded at boot with /boot/loader.conf or the `GEOM_LABEL` kernel option is present, the device node may change without any ill effect on the system.

File systems may also be created with a default label by using the `-L` flag with `newfs`. Refer to newfs(8) for more information.

The following command can be used to destroy the label:

```
# glabel destroy home
```

The following example shows how to label the partitions of a boot disk.

Example 32. Labeling Partitions on the Boot Disk

By permanently labeling the partitions on the boot disk, the system should be able to continue to boot normally, even if the disk is moved to another controller or transferred to a different system. For this example, it is assumed that a single ATA disk is used, which is currently recognized by the system as ad0. It is also assumed that the standard FreeBSD partition scheme is used, with /, /var, /usr and /tmp, as well as a swap partition.

Reboot the system, and at the loader(8) prompt, press `4` to boot into single user mode. Then enter the following commands:

```
# glabel label rootfs /dev/ad0s1a
GEOM_LABEL: Label for provider /dev/ad0s1a is label/rootfs
# glabel label var /dev/ad0s1d
GEOM_LABEL: Label for provider /dev/ad0s1d is label/var
# glabel label usr /dev/ad0s1f
GEOM_LABEL: Label for provider /dev/ad0s1f is label/usr
# glabel label tmp /dev/ad0s1e
GEOM_LABEL: Label for provider /dev/ad0s1e is label/tmp
# glabel label swap /dev/ad0s1b
GEOM_LABEL: Label for provider /dev/ad0s1b is label/swap
# exit
```

The system will continue with multi-user boot. After the boot completes, edit /etc/fstab and replace the conventional device names, with their respective labels. The final /etc/fstab will look like this:

```
# Device                Mountpoint      FStype  Options         Dump    Pass#
/dev/label/swap         none            swap    sw              0       0
/dev/label/rootfs       /               ufs     rw              1       1
/dev/label/tmp          /tmp            ufs     rw              2       2
/dev/label/usr          /usr            ufs     rw              2       2
/dev/label/var          /var            ufs     rw              2       2
```

The system can now be rebooted. If everything went well, it will come up normally and `mount` will show:

```
# mount
/dev/label/rootfs on / (ufs, local)
devfs on /dev (devfs, local)
/dev/label/tmp on /tmp (ufs, local, soft-updates)
/dev/label/usr on /usr (ufs, local, soft-updates)
/dev/label/var on /var (ufs, local, soft-updates)
```

The glabel(8) class supports a label type for UFS file systems, based on the unique file system id, `ufsid`. These labels may be found in /dev/ufsid and are created automatically during system startup. It is possible to use `ufsid` labels to mount partitions using /etc/fstab. Use `glabel status` to receive a list of file systems and their corresponding `ufsid` labels:

```
% glabel status
                  Name  Status  Components
ufsid/486b6fc38d330916     N/A  ad4s1d
ufsid/486b6fc16926168e     N/A  ad4s1f
```

In the above example, ad4s1d represents /var, while ad4s1f represents /usr. Using the `ufsid` values shown, these partitions may now be mounted with the following entries in /etc/fstab:

```
/dev/ufsid/486b6fc38d330916        /var        ufs        rw        2      2
/dev/ufsid/486b6fc16926168e        /usr        ufs        rw        2      2
```

Any partitions with `ufsid` labels can be mounted in this way, eliminating the need to manually create permanent labels, while still enjoying the benefits of device name independent mounting.

21.8. UFS Journaling Through GEOM

Support for journals on UFS file systems is available on FreeBSD. The implementation is provided through the GEOM subsystem and is configured using `gjournal`. Unlike other file system journaling implementations, the `gjournal` method is block based and not implemented as part of the file system. It is a GEOM extension.

Journaling stores a log of file system transactions, such as changes that make up a complete disk write operation, before meta-data and file writes are committed to the disk. This transaction log can later be replayed to redo file system transactions, preventing file system inconsistencies.

This method provides another mechanism to protect against data loss and inconsistencies of the file system. Unlike Soft Updates, which tracks and enforces meta-data updates, and snapshots, which create an image of the file system, a log is stored in disk space specifically for this task. For better performance, the journal may be stored on another disk. In this configuration, the journal provider or storage device should be listed after the device to enable journaling on.

The GENERIC kernel provides support for `gjournal`. To automatically load the geom_journal.ko kernel module at boot time, add the following line to /boot/loader.conf:

```
geom_journal_load="YES"
```

If a custom kernel is used, ensure the following line is in the kernel configuration file:

```
options GEOM_JOURNAL
```

Once the module is loaded, a journal can be created on a new file system using the following steps. In this example, da4 is a new SCSI disk:

```
# gjournal load
# gjournal label /dev/da4
```

This will load the module and create a /dev/da4.journal device node on /dev/da4.

A UFS file system may now be created on the journaled device, then mounted on an existing mount point:

```
# newfs -O 2 -J /dev/da4.journal
# mount /dev/da4.journal /mnt
```



In the case of several slices, a journal will be created for each individual slice. For instance, if ad4s1 and ad4s2 are both slices, then `gjournal` will create ad4s1.journal and ad4s2.journal.

Journaling may also be enabled on current file systems by using `tunefs`. However, *always* make a backup before attempting to alter an existing file system. In most cases, `gjournal` will fail if it is unable to create the journal, but this does not protect against data loss incurred as a result of misusing `tunefs`. Refer to gjournal(8) and tunefs(8) for more information about these commands.

It is possible to journal the boot disk of a FreeBSD system. Refer to the article Implementing UFS Journaling on a Desktop PC for detailed instructions.

Chapter 22. The Z File System (ZFS)

ZFS is an advanced file system designed to solve major problems found in previous storage subsystem software.

Originally developed at Sun™, ongoing open source ZFS development has moved to the OpenZFS Project.

ZFS has three major design goals:

- Data integrity: All data includes a checksum of the data. ZFS calculates checksums and writes them along with the data. When reading that data later, ZFS recalculates the checksums. If the checksums do not match, meaning detecting one or more data errors, ZFS will attempt to automatically correct errors when ditto-, mirror-, or parity-blocks are available.
- Pooled storage: adding physical storage devices to a pool, and allocating storage space from that shared pool. Space is available to all file systems and volumes, and increases by adding new storage devices to the pool.
- Performance: caching mechanisms provide increased performance. ARC is an advanced memory-based read cache. ZFS provides a second level disk-based read cache with L2ARC, and a disk-based synchronous write cache named ZIL.

A complete list of features and terminology is in ZFS Features and Terminology.

22.1. What Makes ZFS Different

More than a file system, ZFS is fundamentally different from traditional file systems. Combining the traditionally separate roles of volume manager and file system provides ZFS with unique advantages. The file system is now aware of the underlying structure of the disks. Traditional file systems could exist on a single disk alone at a time. If there were two disks then creating two separate file systems was necessary. A traditional hardware RAID configuration avoided this problem by presenting the operating system with a single logical disk made up of the space provided by physical disks on top of which the operating system placed a file system. Even with software RAID solutions like those provided by GEOM, the UFS file system living on top of the RAID believes it's dealing with a single device. ZFS' combination of the volume manager and the file system solves this and allows the creation of file systems that all share a pool of available storage. One big advantage of ZFS' awareness of the physical disk layout is that existing file systems grow automatically when adding extra disks to the pool. This new space then becomes available to the file systems. ZFS can also apply different properties to each file system. This makes it useful to create separate file systems and datasets instead of a single monolithic file system.

22.2. Quick Start Guide

FreeBSD can mount ZFS pools and datasets during system initialization. To enable it, run:

```
# service zfs enable
```

or add this line to /etc/rc.conf:

```
zfs_enable="YES"
```

Then start the service:

```
# service zfs start
```

The examples in this section assume three SCSI disks with the device names da0, da1, and da2. Users of SATA hardware should instead use ada device names.

22.2.1. Single Disk Pool

To create a simple, non-redundant pool using a single disk device:

```
# zpool create example /dev/da0
```

To view the new pool, review the output of `df`:

```
# df
Filesystem  1K-blocks    Used    Avail Capacity  Mounted on
/dev/ad0s1a   2026030  235230  1628718    13%    /
devfs               1       1        0   100%    /dev
/dev/ad0s1d  54098308 1032846 48737598     2%    /usr
example      17547136       0 17547136     0%    /example
```

This output shows creating and mounting of the `example` pool, and that is now accessible as a file system. Create files for users to browse:

```
# cd /example
# ls
# touch testfile
# ls -al
total 4
drwxr-xr-x   2 root  wheel    3 Aug 29 23:15 .
drwxr-xr-x  21 root  wheel  512 Aug 29 23:12 ..
-rw-r--r--   1 root  wheel    0 Aug 29 23:15 testfile
```

This pool is not using any advanced ZFS features and properties yet. To create a dataset on this pool with compression enabled:

```
# zfs create example/compressed
# zfs set compression=gzip example/compressed
```

The `example/compressed` dataset is now a ZFS compressed file system. Try copying some large files to /example/compressed.

Disable compression with:

```
# zfs set compression=off example/compressed
```

To unmount a file system, use `zfs umount` and then verify with `df`:

```
# zfs umount example/compressed
# df
Filesystem  1K-blocks    Used    Avail Capacity  Mounted on
/dev/ad0s1a   2026030  235232  1628716    13%    /
devfs               1       1        0   100%    /dev
/dev/ad0s1d  54098308 1032864 48737580     2%    /usr
example      17547008       0 17547008     0%    /example
```

To re-mount the file system to make it accessible again, use `zfs mount` and verify with `df`:

```
# zfs mount example/compressed
# df
Filesystem         1K-blocks    Used    Avail Capacity  Mounted on
/dev/ad0s1a          2026030  235234  1628714    13%    /
devfs                      1       1        0   100%    /dev
/dev/ad0s1d         54098308 1032864 48737580     2%    /usr
example             17547008       0 17547008     0%    /example
example/compressed  17547008       0 17547008     0%    /example/compressed
```

Running `mount` shows the pool and file systems:

```
# mount
/dev/ad0s1a on / (ufs, local)
devfs on /dev (devfs, local)
/dev/ad0s1d on /usr (ufs, local, soft-updates)
example on /example (zfs, local)
example/compressed on /example/compressed (zfs, local)
```

Use ZFS datasets like any file system after creation. Set other available features on a per-dataset basis when needed. The example below creates a new file system called `data`. It assumes the file system contains important files and configures it to store two copies of each data block.

```
# zfs create example/data
# zfs set copies=2 example/data
```

Use `df` to see the data and space usage:

```
# df
Filesystem         1K-blocks    Used    Avail Capacity  Mounted on
/dev/ad0s1a          2026030  235234  1628714    13%    /
devfs                      1       1        0   100%    /dev
/dev/ad0s1d         54098308 1032864 48737580     2%    /usr
example             17547008       0 17547008     0%    /example
example/compressed  17547008       0 17547008     0%    /example/compressed
example/data        17547008       0 17547008     0%    /example/data
```

Notice that all file systems in the pool have the same available space. Using `df` in these examples shows that the file systems use the space they need and all draw from the same pool. ZFS gets rid of concepts such as volumes and partitions, and allows several file systems to share the same pool.

To destroy the file systems and then the pool that is no longer needed:

```
# zfs destroy example/compressed
# zfs destroy example/data
# zpool destroy example
```

22.2.2. RAID-Z

Disks fail. One way to avoid data loss from disk failure is to use RAID. ZFS supports this feature in its pool design. RAID-Z pools require three or more disks but provide more usable space than mirrored pools.

This example creates a RAID-Z pool, specifying the disks to add to the pool:

```
# zpool create storage raidz da0 da1 da2
```



Sun™ recommends that the number of devices used in a RAID-Z configuration be between three and nine. For environments requiring a single pool consisting of 10 disks or more, consider breaking it up into smaller RAID-Z groups. If two disks are available, ZFS mirroring provides redundancy if required. Refer to zpool(8) for more details.

The previous example created the `storage` zpool. This example makes a new file system called `home` in that pool:

```
# zfs create storage/home
```

Enable compression and store an extra copy of directories and files:

```
# zfs set copies=2 storage/home
# zfs set compression=gzip storage/home
```

To make this the new home directory for users, copy the user data to this directory and create the appropriate symbolic links:

```
# cp -rp /home/* /storage/home
# rm -rf /home /usr/home
# ln -s /storage/home /home
# ln -s /storage/home /usr/home
```

Users data is now stored on the freshly-created /storage/home. Test by adding a new user and logging in as that user.

Create a file system snapshot to roll back to later:

```
# zfs snapshot storage/home@08-30-08
```

ZFS creates snapshots of a dataset, not a single directory or file.

The `@` character is a delimiter between the file system name or the volume name. Before deleting an important directory, back up the file system, then roll back to an earlier snapshot in which the directory still exists:

```
# zfs rollback storage/home@08-30-08
```

To list all available snapshots, run `ls` in the file system's .zfs/snapshot directory. For example, to see the snapshot taken:

```
# ls /storage/home/.zfs/snapshot
```

Write a script to take regular snapshots of user data. Over time, snapshots can use up a lot of disk space. Remove the previous snapshot using the command:

```
# zfs destroy storage/home@08-30-08
```

After testing, make /storage/home the real /home with this command:

```
# zfs set mountpoint=/home storage/home
```

Run `df` and `mount` to confirm that the system now treats the file system as the real /home:

```
# mount
/dev/ad0s1a on / (ufs, local)
devfs on /dev (devfs, local)
/dev/ad0s1d on /usr (ufs, local, soft-updates)
```

```
storage on /storage (zfs, local)
storage/home on /home (zfs, local)
# df
Filesystem   1K-blocks    Used    Avail Capacity  Mounted on
/dev/ad0s1a    2026030  235240  1628708    13%    /
devfs                1       1        0   100%    /dev
/dev/ad0s1d   54098308 1032826 48737618     2%    /usr
storage       26320512       0 26320512     0%    /storage
storage/home  26320512       0 26320512     0%    /home
```

This completes the RAID-Z configuration. Add daily status updates about the created file systems to the nightly periodic(8) runs by adding this line to /etc/periodic.conf:

```
daily_status_zfs_enable="YES"
```

22.2.3. Recovering RAID-Z

Every software RAID has a method of monitoring its `state`. View the status of RAID-Z devices using:

```
# zpool status -x
```

If all pools are Online and everything is normal, the message shows:

```
all pools are healthy
```

If there is a problem, perhaps a disk being in the Offline state, the pool state will look like this:

```
  pool: storage
 state: DEGRADED
status: One or more devices has been taken offline by the administrator.
    Sufficient replicas exist for the pool to continue functioning in a
    degraded state.
action: Online the device using 'zpool online' or replace the device with
    'zpool replace'.
 scrub: none requested
config:

    NAME        STATE     READ WRITE CKSUM
    storage     DEGRADED     0     0     0
      raidz1    DEGRADED     0     0     0
        da0     ONLINE       0     0     0
        da1     OFFLINE      0     0     0
        da2     ONLINE       0     0     0

errors: No known data errors
```

"OFFLINE" shows the administrator took da1 offline using:

```
# zpool offline storage da1
```

Power down the computer now and replace da1. Power up the computer and return da1 to the pool:

```
# zpool replace storage da1
```

Next, check the status again, this time without `-x` to display all pools:

```
# zpool status storage
 pool: storage
 state: ONLINE
 scrub: resilver completed with 0 errors on Sat Aug 30 19:44:11 2008
config:

    NAME        STATE     READ WRITE CKSUM
    storage     ONLINE       0     0     0
      raidz1    ONLINE       0     0     0
        da0     ONLINE       0     0     0
        da1     ONLINE       0     0     0
        da2     ONLINE       0     0     0

errors: No known data errors
```

In this example, everything is normal.

22.2.4. Data Verification

ZFS uses checksums to verify the integrity of stored data. Creating file systems automatically enables them.



Disabling Checksums is possible but *not* recommended! Checksums take little storage space and provide data integrity. Most ZFS features will not work properly with checksums disabled. Disabling these checksums will not increase performance noticeably.

Verifying the data checksums (called *scrubbing*) ensures integrity of the `storage` pool with:

```
# zpool scrub storage
```

The duration of a scrub depends on the amount of data stored. Larger amounts of data will take proportionally longer to verify. Since scrubbing is I/O intensive, ZFS allows a single scrub to run at a time. After scrubbing completes, view the status with `zpool status`:

```
# zpool status storage
 pool: storage
 state: ONLINE
 scrub: scrub completed with 0 errors on Sat Jan 26 19:57:37 2013
config:

    NAME        STATE     READ WRITE CKSUM
    storage     ONLINE       0     0     0
      raidz1    ONLINE       0     0     0
        da0     ONLINE       0     0     0
        da1     ONLINE       0     0     0
        da2     ONLINE       0     0     0

errors: No known data errors
```

Displaying the completion date of the last scrubbing helps decide when to start another. Routine scrubs help protect data from silent corruption and ensure the integrity of the pool.

Refer to zfs(8) and zpool(8) for other ZFS options.

22.3. `zpool` Administration

ZFS administration uses two main utilities. The `zpool` utility controls the operation of the pool and allows adding, removing, replacing, and managing disks. The `zfs` utility allows creating, destroying, and managing datasets, both file systems and volumes.

22.3.1. Creating and Destroying Storage Pools

Creating a ZFS storage pool requires permanent decisions, as the pool structure cannot change after creation. The most important decision is which types of vdevs to group the physical disks into. See the list of vdev types for details about the possible options. After creating the pool, most vdev types do not allow adding disks to the vdev. The exceptions are mirrors, which allow adding new disks to the vdev, and stripes, which upgrade to mirrors by attaching a new disk to the vdev. Although adding new vdevs expands a pool, the pool layout cannot change after pool creation. Instead, back up the data, destroy the pool, and recreate it.

Create a simple mirror pool:

```
# zpool create mypool mirror /dev/ada1 /dev/ada2
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
```

```
            ada1    ONLINE       0     0     0
            ada2    ONLINE       0     0     0

errors: No known data errors
```

To create more than one vdev with a single command, specify groups of disks separated by the vdev type keyword, `mirror` in this example:

```
# zpool create mypool mirror /dev/ada1 /dev/ada2 mirror /dev/ada3 /dev/ada4
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada1    ONLINE       0     0     0
            ada2    ONLINE       0     0     0
          mirror-1  ONLINE       0     0     0
            ada3    ONLINE       0     0     0
            ada4    ONLINE       0     0     0

errors: No known data errors
```

Pools can also use partitions rather than whole disks. Putting ZFS in a separate partition allows the same disk to have other partitions for other purposes. In particular, it allows adding partitions with bootcode and file systems needed for booting. This allows booting from disks that are also members of a pool. ZFS adds no performance penalty on FreeBSD when using a partition rather than a whole disk. Using partitions also allows the administrator to *under-provision* the disks, using less than the full capacity. If a future replacement disk of the same nominal size as the original actually has a slightly smaller capacity, the smaller partition will still fit, using the replacement disk.

Create a RAID-Z2 pool using partitions:

```
# zpool create mypool raidz2 /dev/ada0p3 /dev/ada1p3 /dev/ada2p3 /dev/ada3p3
/dev/ada4p3 /dev/ada5p3
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          raidz2-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
```

```
            ada1p3  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0
            ada3p3  ONLINE       0     0     0
            ada4p3  ONLINE       0     0     0
            ada5p3  ONLINE       0     0     0

errors: No known data errors
```

Destroy a pool that is no longer needed to reuse the disks. Destroying a pool requires unmounting the file systems in that pool first. If any dataset is in use, the unmount operation fails without destroying the pool. Force the pool destruction with `-f`. This can cause undefined behavior in applications which had open files on those datasets.

22.3.2. Adding and Removing Devices

Two ways exist for adding disks to a pool: attaching a disk to an existing vdev with `zpool attach`, or adding vdevs to the pool with `zpool add`. Some vdev types allow adding disks to the vdev after creation.

A pool created with a single disk lacks redundancy. It can detect corruption but can not repair it, because there is no other copy of the data. The copies property may be able to recover from a small failure such as a bad sector, but does not provide the same level of protection as mirroring or RAID-Z. Starting with a pool consisting of a single disk vdev, use `zpool attach` to add a new disk to the vdev, creating a mirror. Also use `zpool attach` to add new disks to a mirror group, increasing redundancy and read performance. When partitioning the disks used for the pool, replicate the layout of the first disk on to the second. Use `gpart backup` and `gpart restore` to make this process easier.

Upgrade the single disk (stripe) vdev ada0p3 to a mirror by attaching ada1p3:

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          ada0p3    ONLINE       0     0     0

errors: No known data errors
# zpool attach mypool ada0p3 ada1p3
Make sure to wait until resilvering finishes before rebooting.

If you boot from pool 'mypool', you may need to update boot code on newly attached
disk _ada1p3_.

Assuming you use GPT partitioning and _da0_ is your new boot disk you may use the
following command:
```

```
        gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 da0
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada1
bootcode written to ada1
# zpool status
  pool: mypool
 state: ONLINE
status: One or more devices is currently being resilvered.  The pool will
        continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
  scan: resilver in progress since Fri May 30 08:19:19 2014
        527M scanned out of 781M at 47.9M/s, 0h0m to go
        527M resilvered, 67.53% done
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0  (resilvering)

errors: No known data errors
# zpool status
  pool: mypool
 state: ONLINE
  scan: resilvered 781M in 0h0m with 0 errors on Fri May 30 08:15:58 2014
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0

errors: No known data errors
```

When adding disks to the existing vdev is not an option, as for RAID-Z, an alternative method is to add another vdev to the pool. Adding vdevs provides higher performance by distributing writes across the vdevs. Each vdev provides its own redundancy. Mixing vdev types like `mirror` and `RAID-Z` is possible but discouraged. Adding a non-redundant vdev to a pool containing mirror or RAID-Z vdevs risks the data on the entire pool. Distributing writes means a failure of the non-redundant disk will result in the loss of a fraction of every block written to the pool.

ZFS stripes data across each of the vdevs. For example, with two mirror vdevs, this is effectively a RAID 10 that stripes writes across two sets of mirrors. ZFS allocates space so that each vdev reaches 100% full at the same time. Having vdevs with different amounts of free space will lower performance, as more data writes go to the less full vdev.

When attaching new devices to a boot pool, remember to update the bootcode.

Attach a second mirror group (ada2p3 and ada3p3) to the existing mirror:

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: resilvered 781M in 0h0m with 0 errors on Fri May 30 08:19:35 2014
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0

errors: No known data errors
# zpool add mypool mirror ada2p3 ada3p3
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada2
bootcode written to ada2
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada3
bootcode written to ada3
# zpool status
  pool: mypool
 state: ONLINE
  scan: scrub repaired 0 in 0h0m with 0 errors on Fri May 30 08:29:51 2014
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0
          mirror-1  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0
            ada3p3  ONLINE       0     0     0

errors: No known data errors
```

Removing vdevs from a pool is impossible and removal of disks from a mirror is exclusive if there is enough remaining redundancy. If a single disk remains in a mirror group, that group ceases to be a mirror and becomes a stripe, risking the entire pool if that remaining disk fails.

Remove a disk from a three-way mirror group:

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: scrub repaired 0 in 0h0m with 0 errors on Fri May 30 08:29:51 2014
config:
```

```
        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0

errors: No known data errors
# zpool detach mypool ada2p3
# zpool status
  pool: mypool
 state: ONLINE
  scan: scrub repaired 0 in 0h0m with 0 errors on Fri May 30 08:29:51 2014
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0

errors: No known data errors
```

22.3.3. Checking the Status of a Pool

Pool status is important. If a drive goes offline or ZFS detects a read, write, or checksum error, the corresponding error count increases. The `status` output shows the configuration and status of each device in the pool and the status of the entire pool. Actions to take and details about the last `scrub` are also shown.

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: scrub repaired 0 in 2h25m with 0 errors on Sat Sep 14 04:25:50 2013
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          raidz2-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0
            ada3p3  ONLINE       0     0     0
            ada4p3  ONLINE       0     0     0
            ada5p3  ONLINE       0     0     0

errors: No known data errors
```

22.3.4. Clearing Errors

When detecting an error, ZFS increases the read, write, or checksum error counts. Clear the error message and reset the counts with `zpool clear mypool`. Clearing the error state can be important for automated scripts that alert the administrator when the pool encounters an error. Without clearing old errors, the scripts may fail to report further errors.

22.3.5. Replacing a Functioning Device

It may be desirable to replace one disk with a different disk. When replacing a working disk, the process keeps the old disk online during the replacement. The pool never enters a degraded state, reducing the risk of data loss. Running `zpool replace` copies the data from the old disk to the new one. After the operation completes, ZFS disconnects the old disk from the vdev. If the new disk is larger than the old disk, it may be possible to grow the zpool, using the new space. See Growing a Pool.

Replace a functioning device in the pool:

```
# zpool status
  pool: mypool
 state: ONLINE
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0

errors: No known data errors
# zpool replace mypool ada1p3 ada2p3
Make sure to wait until resilvering finishes before rebooting.

When booting from the pool 'zroot', update the boot code on the newly attached disk
'ada2p3'.

Assuming GPT partitioning is used and [.filename]#da0# is the new boot disk, use the
following command:

        gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 da0
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada2
# zpool status
  pool: mypool
 state: ONLINE
status: One or more devices is currently being resilvered.  The pool will
        continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
  scan: resilver in progress since Mon Jun  2 14:21:35 2014
        604M scanned out of 781M at 46.5M/s, 0h0m to go
```

```
        604M resilvered, 77.39% done
config:

        NAME             STATE     READ WRITE CKSUM
        mypool           ONLINE       0     0     0
          mirror-0       ONLINE       0     0     0
            ada0p3       ONLINE       0     0     0
            replacing-1  ONLINE       0     0     0
              ada1p3     ONLINE       0     0     0
              ada2p3     ONLINE       0     0     0  (resilvering)

errors: No known data errors
# zpool status
  pool: mypool
 state: ONLINE
  scan: resilvered 781M in 0h0m with 0 errors on Mon Jun  2 14:21:52 2014
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0

errors: No known data errors
```

22.3.6. Dealing with Failed Devices

When a disk in a pool fails, the vdev to which the disk belongs enters the degraded state. The data is still available, but with reduced performance because ZFS computes missing data from the available redundancy. To restore the vdev to a fully functional state, replace the failed physical device. ZFS is then instructed to begin the resilver operation. ZFS recomputes data on the failed device from available redundancy and writes it to the replacement device. After completion, the vdev returns to online status.

If the vdev does not have any redundancy, or if devices have failed and there is not enough redundancy to compensate, the pool enters the faulted state. Unless enough devices can reconnect the pool becomes inoperative requiring a data restore from backups.

When replacing a failed disk, the name of the failed disk changes to the GUID of the new disk. A new device name parameter for `zpool replace` is not required if the replacement device has the same device name.

Replace a failed disk using `zpool replace`:

```
# zpool status
  pool: mypool
 state: DEGRADED
status: One or more devices could not be opened.  Sufficient replicas exist for
```

```
        the pool to continue functioning in a degraded state.
action: Attach the missing device and online it using 'zpool online'.
   see: http://illumos.org/msg/ZFS-8000-2Q
  scan: none requested
config:

        NAME                     STATE     READ WRITE CKSUM
        mypool                   DEGRADED     0     0     0
          mirror-0               DEGRADED     0     0     0
            ada0p3               ONLINE       0     0     0
            316502962686821739   UNAVAIL      0     0     0  was /dev/ada1p3

errors: No known data errors
# zpool replace mypool 316502962686821739 ada2p3
# zpool status
  pool: mypool
 state: DEGRADED
status: One or more devices is currently being resilvered.  The pool will
        continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
  scan: resilver in progress since Mon Jun  2 14:52:21 2014
        641M scanned out of 781M at 49.3M/s, 0h0m to go
        640M resilvered, 82.04% done
config:

        NAME                       STATE     READ WRITE CKSUM
        mypool                     DEGRADED     0     0     0
          mirror-0                 DEGRADED     0     0     0
            ada0p3                 ONLINE       0     0     0
            replacing-1            UNAVAIL      0     0     0
              15732067398082357289 UNAVAIL      0     0     0  was /dev/ada1p3/old
              ada2p3               ONLINE       0     0     0  (resilvering)

errors: No known data errors
# zpool status
  pool: mypool
 state: ONLINE
  scan: resilvered 781M in 0h0m with 0 errors on Mon Jun  2 14:52:38 2014
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0

errors: No known data errors
```

22.3.7. Scrubbing a Pool

Routinely scrub pools, ideally at least once every month. The `scrub` operation is disk-intensive and will reduce performance while running. Avoid high-demand periods when scheduling `scrub` or use `vfs.zfs.scrub_delay` to adjust the relative priority of the `scrub` to keep it from slowing down other workloads.

```
# zpool scrub mypool
# zpool status
  pool: mypool
 state: ONLINE
  scan: scrub in progress since Wed Feb 19 20:52:54 2014
        116G scanned out of 8.60T at 649M/s, 3h48m to go
        0 repaired, 1.32% done
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          raidz2-0  ONLINE       0     0     0
            ada0p3  ONLINE       0     0     0
            ada1p3  ONLINE       0     0     0
            ada2p3  ONLINE       0     0     0
            ada3p3  ONLINE       0     0     0
            ada4p3  ONLINE       0     0     0
            ada5p3  ONLINE       0     0     0

errors: No known data errors
```

To cancel a scrub operation if needed, run `zpool scrub -s mypool`.

22.3.8. Self-Healing

The checksums stored with data blocks enable the file system to *self-heal*. This feature will automatically repair data whose checksum does not match the one recorded on another device that is part of the storage pool. For example, a mirror configuration with two disks where one drive is starting to malfunction and cannot properly store the data any more. This is worse when the data was not accessed for a long time, as with long term archive storage. Traditional file systems need to run commands that check and repair the data like fsck(8). These commands take time, and in severe cases, an administrator has to decide which repair operation to perform. When ZFS detects a data block with a mismatched checksum, it tries to read the data from the mirror disk. If that disk can provide the correct data, ZFS will give that to the application and correct the data on the disk with the wrong checksum. This happens without any interaction from a system administrator during normal pool operation.

The next example shows this self-healing behavior by creating a mirrored pool of disks /dev/ada0 and /dev/ada1.

```
# zpool create healer mirror /dev/ada0 /dev/ada1
```

```
# zpool status healer
  pool: healer
 state: ONLINE
  scan: none requested
config:

    NAME        STATE     READ WRITE CKSUM
    healer      ONLINE       0     0     0
      mirror-0  ONLINE       0     0     0
        ada0    ONLINE       0     0     0
        ada1    ONLINE       0     0     0

errors: No known data errors
# zpool list
NAME     SIZE  ALLOC   FREE   CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP  HEALTH  ALTROOT
healer   960M  92.5K   960M         -         -     0%     0%  1.00x  ONLINE  -
```

Copy some important data to the pool to protect from data errors using the self-healing feature and create a checksum of the pool for later comparison.

```
# cp /some/important/data /healer
# zfs list
NAME     SIZE  ALLOC   FREE    CAP  DEDUP  HEALTH  ALTROOT
healer   960M  67.7M   892M     7%  1.00x  ONLINE  -
# sha1 /healer > checksum.txt
# cat checksum.txt
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
```

Simulate data corruption by writing random data to the beginning of one of the disks in the mirror. To keep ZFS from healing the data when detected, export the pool before the corruption and import it again afterwards.



This is a dangerous operation that can destroy vital data, shown here for demonstration alone. **Do not try** it during normal operation of a storage pool. Nor should this intentional corruption example run on any disk with a file system not using ZFS on another partition in it. Do not use any other disk device names other than the ones that are part of the pool. Ensure proper backups of the pool exist and test them before running the command!

```
# zpool export healer
# dd if=/dev/random of=/dev/ada1 bs=1m count=200
200+0 records in
200+0 records out
209715200 bytes transferred in 62.992162 secs (3329227 bytes/sec)
# zpool import healer
```

The pool status shows that one device has experienced an error. Note that applications reading data

from the pool did not receive any incorrect data. ZFS provided data from the ada0 device with the correct checksums. To find the device with the wrong checksum, look for one whose `CKSUM` column contains a nonzero value.

```
# zpool status healer
    pool: healer
   state: ONLINE
  status: One or more devices has experienced an unrecoverable error.  An
          attempt was made to correct the error.  Applications are unaffected.
  action: Determine if the device needs to be replaced, and clear the errors
          using 'zpool clear' or replace the device with 'zpool replace'.
     see: http://illumos.org/msg/ZFS-8000-4J
    scan: none requested
  config:

      NAME        STATE     READ WRITE CKSUM
      healer      ONLINE       0     0     0
        mirror-0  ONLINE       0     0     0
          ada0    ONLINE       0     0     0
          ada1    ONLINE       0     0     1

errors: No known data errors
```

ZFS detected the error and handled it by using the redundancy present in the unaffected ada0 mirror disk. A checksum comparison with the original one will reveal whether the pool is consistent again.

```
# sha1 /healer >> checksum.txt
# cat checksum.txt
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
```

Generate checksums before and after the intentional tampering while the pool data still matches. This shows how ZFS is capable of detecting and correcting any errors automatically when the checksums differ. Note this is possible with enough redundancy present in the pool. A pool consisting of a single device has no self-healing capabilities. That is also the reason why checksums are so important in ZFS; do not disable them for any reason. ZFS requires no fsck(8) or similar file system consistency check program to detect and correct this, and keeps the pool available while there is a problem. A scrub operation is now required to overwrite the corrupted data on ada1.

```
# zpool scrub healer
# zpool status healer
  pool: healer
 state: ONLINE
status: One or more devices has experienced an unrecoverable error.  An
            attempt was made to correct the error.  Applications are unaffected.
action: Determine if the device needs to be replaced, and clear the errors
```

```
            using 'zpool clear' or replace the device with 'zpool replace'.
   see: http://illumos.org/msg/ZFS-8000-4J
  scan: scrub in progress since Mon Dec 10 12:23:30 2012
        10.4M scanned out of 67.0M at 267K/s, 0h3m to go
        9.63M repaired, 15.56% done
config:

    NAME        STATE     READ WRITE CKSUM
    healer      ONLINE       0     0     0
      mirror-0  ONLINE       0     0     0
        ada0    ONLINE       0     0     0
        ada1    ONLINE       0     0   627  (repairing)

errors: No known data errors
```

The scrub operation reads data from ada0 and rewrites any data with a wrong checksum on ada1, shown by the `(repairing)` output from `zpool status`. After the operation is complete, the pool status changes to:

```
# zpool status healer
  pool: healer
 state: ONLINE
status: One or more devices has experienced an unrecoverable error.  An
        attempt was made to correct the error.  Applications are unaffected.
action: Determine if the device needs to be replaced, and clear the errors
             using 'zpool clear' or replace the device with 'zpool replace'.
   see: http://illumos.org/msg/ZFS-8000-4J
  scan: scrub repaired 66.5M in 0h2m with 0 errors on Mon Dec 10 12:26:25 2012
config:

    NAME        STATE     READ WRITE CKSUM
    healer      ONLINE       0     0     0
      mirror-0  ONLINE       0     0     0
        ada0    ONLINE       0     0     0
        ada1    ONLINE       0     0 2.72K

errors: No known data errors
```

After the scrubbing operation completes with all the data synchronized from ada0 to ada1, clear the error messages from the pool status by running `zpool clear`.

```
# zpool clear healer
# zpool status healer
  pool: healer
 state: ONLINE
  scan: scrub repaired 66.5M in 0h2m with 0 errors on Mon Dec 10 12:26:25 2012
config:
```

```
    NAME        STATE     READ WRITE CKSUM
    healer      ONLINE       0     0     0
      mirror-0  ONLINE       0     0     0
        ada0    ONLINE       0     0     0
        ada1    ONLINE       0     0     0

errors: No known data errors
```

The pool is now back to a fully working state, with all error counts now zero.

22.3.9. Growing a Pool

The smallest device in each vdev limits the usable size of a redundant pool. Replace the smallest device with a larger device. After completing a replace or resilver operation, the pool can grow to use the capacity of the new device. For example, consider a mirror of a 1 TB drive and a 2 TB drive. The usable space is 1 TB. When replacing the 1 TB drive with another 2 TB drive, the resilvering process copies the existing data onto the new drive. As both of the devices now have 2 TB capacity, the mirror's available space grows to 2 TB.

Start expansion by using `zpool online -e` on each device. After expanding all devices, the extra space becomes available to the pool.

22.3.10. Importing and Exporting Pools

Export pools before moving them to another system. ZFS unmounts all datasets, marking each device as exported but still locked to prevent use by other disks. This allows pools to be *imported* on other machines, other operating systems that support ZFS, and even different hardware architectures (with some caveats, see zpool(8)). When a dataset has open files, use `zpool export -f` to force exporting the pool. Use this with caution. The datasets are forcibly unmounted, potentially resulting in unexpected behavior by the applications which had open files on those datasets.

Export a pool that is not in use:

```
# zpool export mypool
```

Importing a pool automatically mounts the datasets. If this is undesired behavior, use `zpool import -N` to prevent it. `zpool import -o` sets temporary properties for this specific import. `zpool import altroot=` allows importing a pool with a base mount point instead of the root of the file system. If the pool was last used on a different system and was not properly exported, force the import using `zpool import -f`. `zpool import -a` imports all pools that do not appear to be in use by another system.

List all available pools for import:

```
# zpool import
   pool: mypool
     id: 9930174748043525076
  state: ONLINE
```

```
 action: The pool can be imported using its name or numeric identifier.
 config:

        mypool      ONLINE
          ada2p3    ONLINE
```

Import the pool with an alternative root directory:

```
# zpool import -o altroot=/mnt mypool
# zfs list
zfs list
NAME                 USED  AVAIL  REFER  MOUNTPOINT
mypool               110K  47.0G    31K  /mnt/mypool
```

22.3.11. Upgrading a Storage Pool

After upgrading FreeBSD, or if importing a pool from a system using an older version, manually upgrade the pool to the latest ZFS version to support newer features. Consider whether the pool may ever need importing on an older system before upgrading. Upgrading is a one-way process. Upgrade older pools is possible, but downgrading pools with newer features is not.

Upgrade a v28 pool to support `Feature Flags`:

```
# zpool status
  pool: mypool
 state: ONLINE
status: The pool is formatted using a legacy on-disk format.  The pool can
        still be used, but some features are unavailable.
action: Upgrade the pool using 'zpool upgrade'.  Once this is done, the
        pool will no longer be accessible on software that does not support feat
        flags.
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
        ada0    ONLINE       0     0     0
        ada1    ONLINE       0     0     0

errors: No known data errors
# zpool upgrade
This system supports ZFS pool feature flags.

The following pools are formatted with legacy version numbers and are upgraded to use
feature flags.
After being upgraded, these pools will no longer be accessible by software that does
not support feature flags.
```

```
VER  POOL
---  ------------
28   mypool

Use 'zpool upgrade -v' for a list of available legacy versions.
Every feature flags pool has all supported features enabled.
# zpool upgrade mypool
This system supports ZFS pool feature flags.

Successfully upgraded 'mypool' from version 28 to feature flags.
Enabled the following features on 'mypool':
  async_destroy
  empty_bpobj
  lz4_compress
  multi_vdev_crash_dump
```

The newer features of ZFS will not be available until `zpool upgrade` has completed. Use `zpool upgrade -v` to see what new features the upgrade provides, as well as which features are already supported.

Upgrade a pool to support new feature flags:

```
# zpool status
  pool: mypool
 state: ONLINE
status: Some supported features are not enabled on the pool. The pool can
        still be used, but some features are unavailable.
action: Enable all features using 'zpool upgrade'. Once this is done,
        the pool may no longer be accessible by software that does not support
        the features. See zpool-features(7) for details.
  scan: none requested
config:

        NAME        STATE     READ WRITE CKSUM
        mypool      ONLINE       0     0     0
          mirror-0  ONLINE       0     0     0
        ada0    ONLINE       0     0     0
        ada1    ONLINE       0     0     0

errors: No known data errors
# zpool upgrade
This system supports ZFS pool feature flags.

All pools are formatted using feature flags.

Some supported features are not enabled on the following pools. Once a
feature is enabled the pool may become incompatible with software
that does not support the feature. See zpool-features(7) for details.
```

```
POOL  FEATURE
---------------
zstore
      multi_vdev_crash_dump
      spacemap_histogram
      enabled_txg
      hole_birth
      extensible_dataset
      bookmarks
      filesystem_limits
# zpool upgrade mypool
This system supports ZFS pool feature flags.

Enabled the following features on 'mypool':
  spacemap_histogram
  enabled_txg
  hole_birth
  extensible_dataset
  bookmarks
  filesystem_limits
```



Update the boot code on systems that boot from a pool to support the new pool version. Use `gpart bootcode` on the partition that contains the boot code. Two types of bootcode are available, depending on way the system boots: GPT (the most common option) and EFI (for more modern systems).

For legacy boot using GPT, use the following command:

```
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada1
```

For systems using EFI to boot, execute the following command:

```
# mount -t msdosfs /dev/ada1p1 /boot/efi
# cp /boot/loader.efi /boot/efi/efi/boot/BOOTX64.efi
# cp /boot/loader.efi /boot/efi/efi/freebsd/loader.efi
```

These instructions apply to amd64 systems. Depending on the architecture, the file has a different name. Refer to uefi(8) to find the names of the architecture-specific files.

Apply the bootcode to all bootable disks in the pool. See gpart(8) for more information.

22.3.12. Displaying Recorded Pool History

ZFS records commands that change the pool, including creating datasets, changing properties, or replacing a disk. Reviewing history about a pool's creation is useful, as is checking which user

performed a specific action and when. History is not kept in a log file, but is part of the pool itself. The command to review this history is aptly named `zpool history`:

```
# zpool history
History for 'tank':
2013-02-26.23:02:35 zpool create tank mirror /dev/ada0 /dev/ada1
2013-02-27.18:50:58 zfs set atime=off tank
2013-02-27.18:51:09 zfs set checksum=fletcher4 tank
2013-02-27.18:51:18 zfs create tank/backup
```

The output shows `zpool` and `zfs` commands altering the pool in some way along with a timestamp. Commands like `zfs list` are not included. When specifying no pool name, ZFS displays history of all pools.

`zpool history` can show even more information when providing the options `-i` or `-l`. `-i` displays user-initiated events as well as internally logged ZFS events.

```
# zpool history -i
History for 'tank':
2013-02-26.23:02:35 [internal pool create txg:5] pool spa 28; zfs spa 28; zpl 5;uts
9.1-RELEASE 901000 amd64
2013-02-27.18:50:53 [internal property set txg:50] atime=0 dataset = 21
2013-02-27.18:50:58 zfs set atime=off tank
2013-02-27.18:51:04 [internal property set txg:53] checksum=7 dataset = 21
2013-02-27.18:51:09 zfs set checksum=fletcher4 tank
2013-02-27.18:51:13 [internal create txg:55] dataset = 39
2013-02-27.18:51:18 zfs create tank/backup
```

Show more details by adding `-l`. Showing history records in a long format, including information like the name of the user who issued the command and the hostname on which the change happened.

```
# zpool history -l
History for 'tank':
2013-02-26.23:02:35 zpool create tank mirror /dev/ada0 /dev/ada1 [user 0 (root) on
:global]
2013-02-27.18:50:58 zfs set atime=off tank [user 0 (root) on myzfsbox:global]
2013-02-27.18:51:09 zfs set checksum=fletcher4 tank [user 0 (root) on myzfsbox:global]
2013-02-27.18:51:18 zfs create tank/backup [user 0 (root) on myzfsbox:global]
```

The output shows that the `root` user created the mirrored pool with disks /dev/ada0 and /dev/ada1. The hostname `myzfsbox` is also shown in the commands after the pool's creation. The hostname display becomes important when exporting the pool from one system and importing on another. It's possible to distinguish the commands issued on the other system by the hostname recorded for each command.

Combine both options to `zpool history` to give the most detailed information possible for any given

pool. Pool history provides valuable information when tracking down the actions performed or when needing more detailed output for debugging.

22.3.13. Performance Monitoring

A built-in monitoring system can display pool I/O statistics in real time. It shows the amount of free and used space on the pool, read and write operations performed per second, and I/O bandwidth used. By default, ZFS monitors and displays all pools in the system. Provide a pool name to limit monitoring to that pool. A basic example:

```
# zpool iostat
               capacity     operations    bandwidth
pool        alloc   free   read  write   read  write
----------  -----  -----  -----  -----  -----  -----
data         288G  1.53T      2     11  11.3K  57.1K
```

To continuously see I/O activity, specify a number as the last parameter, indicating an interval in seconds to wait between updates. The next statistic line prints after each interval. Press `Ctrl` + `C` to stop this continuous monitoring. Give a second number on the command line after the interval to specify the total number of statistics to display.

Display even more detailed I/O statistics with `-v`. Each device in the pool appears with a statistics line. This is useful for seeing read and write operations performed on each device, and can help determine if any individual device is slowing down the pool. This example shows a mirrored pool with two devices:

```
# zpool iostat -v
                           capacity     operations    bandwidth
pool                     alloc   free   read  write   read  write
-----------------------  -----  -----  -----  -----  -----  -----
data                      288G  1.53T      2     12  9.23K  61.5K
  mirror                  288G  1.53T      2     12  9.23K  61.5K
    ada1                     -      -      0      4  5.61K  61.7K
    ada2                     -      -      1      4  5.04K  61.7K
-----------------------  -----  -----  -----  -----  -----  -----
```

22.3.14. Splitting a Storage Pool

ZFS can split a pool consisting of one or more mirror vdevs into two pools. Unless otherwise specified, ZFS detaches the last member of each mirror and creates a new pool containing the same data. Be sure to make a dry run of the operation with `-n` first. This displays the details of the requested operation without actually performing it. This helps confirm that the operation will do what the user intends.

22.4. `zfs` Administration

The `zfs` utility can create, destroy, and manage all existing ZFS datasets within a pool. To manage the pool itself, use `zpool`.

22.4.1. Creating and Destroying Datasets

Unlike traditional disks and volume managers, space in ZFS is *not* preallocated. With traditional file systems, after partitioning and assigning the space, there is no way to add a new file system without adding a new disk. With ZFS, creating new file systems is possible at any time. Each *dataset* has properties including features like compression, deduplication, caching, and quotas, as well as other useful properties like readonly, case sensitivity, network file sharing, and a mount point. Nesting datasets within each other is possible and child datasets will inherit properties from their ancestors. Delegate, replicate, snapshot, jail allows administering and destroying each dataset as a unit. Creating a separate dataset for each different type or set of files has advantages. The drawbacks to having a large number of datasets are that some commands like `zfs list` will be slower, and that mounting of hundreds or even thousands of datasets will slow the FreeBSD boot process.

Create a new dataset and enable LZ4 compression on it:

```
# zfs list
NAME                  USED  AVAIL  REFER  MOUNTPOINT
mypool                781M  93.2G   144K  none
mypool/ROOT           777M  93.2G   144K  none
mypool/ROOT/default   777M  93.2G   777M  /
mypool/tmp            176K  93.2G   176K  /tmp
mypool/usr            616K  93.2G   144K  /usr
mypool/usr/home       184K  93.2G   184K  /usr/home
mypool/usr/ports      144K  93.2G   144K  /usr/ports
mypool/usr/src        144K  93.2G   144K  /usr/src
mypool/var           1.20M  93.2G   608K  /var
mypool/var/crash      148K  93.2G   148K  /var/crash
mypool/var/log        178K  93.2G   178K  /var/log
mypool/var/mail       144K  93.2G   144K  /var/mail
mypool/var/tmp        152K  93.2G   152K  /var/tmp
# zfs create -o compress=lz4 mypool/usr/mydataset
# zfs list
NAME                   USED  AVAIL  REFER  MOUNTPOINT
mypool                 781M  93.2G   144K  none
mypool/ROOT            777M  93.2G   144K  none
mypool/ROOT/default    777M  93.2G   777M  /
mypool/tmp             176K  93.2G   176K  /tmp
mypool/usr             704K  93.2G   144K  /usr
mypool/usr/home        184K  93.2G   184K  /usr/home
mypool/usr/mydataset  87.5K  93.2G  87.5K  /usr/mydataset
mypool/usr/ports       144K  93.2G   144K  /usr/ports
mypool/usr/src         144K  93.2G   144K  /usr/src
mypool/var            1.20M  93.2G   610K  /var
```

```
mypool/var/crash       148K  93.2G   148K  /var/crash
mypool/var/log         178K  93.2G   178K  /var/log
mypool/var/mail        144K  93.2G   144K  /var/mail
mypool/var/tmp         152K  93.2G   152K  /var/tmp
```

Destroying a dataset is much quicker than deleting the files on the dataset, as it does not involve scanning the files and updating the corresponding metadata.

Destroy the created dataset:

```
# zfs list
NAME                   USED  AVAIL  REFER  MOUNTPOINT
mypool                 880M  93.1G   144K  none
mypool/ROOT            777M  93.1G   144K  none
mypool/ROOT/default    777M  93.1G   777M  /
mypool/tmp             176K  93.1G   176K  /tmp
mypool/usr             101M  93.1G   144K  /usr
mypool/usr/home        184K  93.1G   184K  /usr/home
mypool/usr/mydataset   100M  93.1G   100M  /usr/mydataset
mypool/usr/ports       144K  93.1G   144K  /usr/ports
mypool/usr/src         144K  93.1G   144K  /usr/src
mypool/var            1.20M  93.1G   610K  /var
mypool/var/crash       148K  93.1G   148K  /var/crash
mypool/var/log         178K  93.1G   178K  /var/log
mypool/var/mail        144K  93.1G   144K  /var/mail
mypool/var/tmp         152K  93.1G   152K  /var/tmp
# zfs destroy mypool/usr/mydataset
# zfs list
NAME                  USED  AVAIL  REFER  MOUNTPOINT
mypool                781M  93.2G   144K  none
mypool/ROOT           777M  93.2G   144K  none
mypool/ROOT/default   777M  93.2G   777M  /
mypool/tmp            176K  93.2G   176K  /tmp
mypool/usr            616K  93.2G   144K  /usr
mypool/usr/home       184K  93.2G   184K  /usr/home
mypool/usr/ports      144K  93.2G   144K  /usr/ports
mypool/usr/src        144K  93.2G   144K  /usr/src
mypool/var           1.21M  93.2G   612K  /var
mypool/var/crash      148K  93.2G   148K  /var/crash
mypool/var/log        178K  93.2G   178K  /var/log
mypool/var/mail       144K  93.2G   144K  /var/mail
mypool/var/tmp        152K  93.2G   152K  /var/tmp
```

In modern versions of ZFS, `zfs destroy` is asynchronous, and the free space might take minutes to appear in the pool. Use `zpool get freeing poolname` to see the `freeing` property, that shows which datasets are having their blocks freed in the background. If there are child datasets, like snapshots or other datasets, destroying the parent is impossible. To destroy a dataset and its children, use `-r` to recursively destroy the dataset and its children. Use `-n -v` to list datasets and snapshots destroyed by this operation, without actually destroy anything. Space reclaimed by destroying snapshots is

also shown.

22.4.2. Creating and Destroying Volumes

A volume is a special dataset type. Rather than mounting as a file system, expose it as a block device under /dev/zvol/poolname/dataset. This allows using the volume for other file systems, to back the disks of a virtual machine, or to make it available to other network hosts using protocols like iSCSI or HAST.

Format a volume with any file system or without a file system to store raw data. To the user, a volume appears to be a regular disk. Putting ordinary file systems on these *zvols* provides features that ordinary disks or file systems do not have. For example, using the compression property on a 250 MB volume allows creation of a compressed FAT file system.

```
# zfs create -V 250m -o compression=on tank/fat32
# zfs list tank
NAME USED AVAIL REFER MOUNTPOINT
tank 258M  670M   31K /tank
# newfs_msdos -F32 /dev/zvol/tank/fat32
# mount -t msdosfs /dev/zvol/tank/fat32 /mnt
# df -h /mnt | grep fat32
Filesystem           Size Used Avail Capacity Mounted on
/dev/zvol/tank/fat32 249M  24k  249M     0%   /mnt
# mount | grep fat32
/dev/zvol/tank/fat32 on /mnt (msdosfs, local)
```

Destroying a volume is much the same as destroying a regular file system dataset. The operation is nearly instantaneous, but it may take minutes to reclaim the free space in the background.

22.4.3. Renaming a Dataset

To change the name of a dataset, use `zfs rename`. To change the parent of a dataset, use this command as well. Renaming a dataset to have a different parent dataset will change the value of those properties inherited from the parent dataset. Renaming a dataset unmounts then remounts it in the new location (inherited from the new parent dataset). To prevent this behavior, use `-u`.

Rename a dataset and move it to be under a different parent dataset:

```
# zfs list
NAME                  USED  AVAIL  REFER  MOUNTPOINT
mypool                780M  93.2G   144K  none
mypool/ROOT           777M  93.2G   144K  none
mypool/ROOT/default   777M  93.2G   777M  /
mypool/tmp            176K  93.2G   176K  /tmp
mypool/usr            704K  93.2G   144K  /usr
mypool/usr/home       184K  93.2G   184K  /usr/home
mypool/usr/mydataset 87.5K  93.2G  87.5K  /usr/mydataset
mypool/usr/ports      144K  93.2G   144K  /usr/ports
```

```
mypool/usr/src         144K  93.2G   144K  /usr/src
mypool/var            1.21M  93.2G   614K  /var
mypool/var/crash       148K  93.2G   148K  /var/crash
mypool/var/log         178K  93.2G   178K  /var/log
mypool/var/mail        144K  93.2G   144K  /var/mail
mypool/var/tmp         152K  93.2G   152K  /var/tmp
# zfs rename mypool/usr/mydataset mypool/var/newname
# zfs list
NAME                  USED  AVAIL  REFER  MOUNTPOINT
mypool                780M  93.2G   144K  none
mypool/ROOT           777M  93.2G   144K  none
mypool/ROOT/default   777M  93.2G   777M  /
mypool/tmp            176K  93.2G   176K  /tmp
mypool/usr            616K  93.2G   144K  /usr
mypool/usr/home       184K  93.2G   184K  /usr/home
mypool/usr/ports      144K  93.2G   144K  /usr/ports
mypool/usr/src        144K  93.2G   144K  /usr/src
mypool/var           1.29M  93.2G   614K  /var
mypool/var/crash      148K  93.2G   148K  /var/crash
mypool/var/log        178K  93.2G   178K  /var/log
mypool/var/mail       144K  93.2G   144K  /var/mail
mypool/var/newname   87.5K  93.2G  87.5K  /var/newname
mypool/var/tmp        152K  93.2G   152K  /var/tmp
```

Renaming snapshots uses the same command. Due to the nature of snapshots, rename cannot change their parent dataset. To rename a recursive snapshot, specify `-r`; this will also rename all snapshots with the same name in child datasets.

```
# zfs list -t snapshot
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool/var/newname@first_snapshot      0      -  87.5K  -
# zfs rename mypool/var/newname@first_snapshot new_snapshot_name
# zfs list -t snapshot
NAME                                   USED  AVAIL  REFER  MOUNTPOINT
mypool/var/newname@new_snapshot_name      0      -  87.5K  -
```

22.4.4. Setting Dataset Properties

Each ZFS dataset has properties that control its behavior. Most properties are automatically inherited from the parent dataset, but can be overridden locally. Set a property on a dataset with `zfs set property=value dataset`. Most properties have a limited set of valid values, `zfs get` will display each possible property and valid values. Using `zfs inherit` reverts most properties to their inherited values. User-defined properties are also possible. They become part of the dataset configuration and provide further information about the dataset or its contents. To distinguish these custom properties from the ones supplied as part of ZFS, use a colon (`:`) to create a custom namespace for the property.

```
# zfs set custom:costcenter=1234 tank
```

```
# zfs get custom:costcenter tank
NAME PROPERTY           VALUE SOURCE
tank custom:costcenter  1234  local
```

To remove a custom property, use `zfs inherit` with `-r`. If the custom property is not defined in any of the parent datasets, this option removes it (but the pool's history still records the change).

```
# zfs inherit -r custom:costcenter tank
# zfs get custom:costcenter tank
NAME    PROPERTY           VALUE              SOURCE
tank    custom:costcenter  -                  -
# zfs get all tank | grep custom:costcenter
#
```

22.4.4.1. Getting and Setting Share Properties

Two commonly used and useful dataset properties are the NFS and SMB share options. Setting these defines if and how ZFS shares datasets on the network. At present, FreeBSD supports setting NFS sharing alone. To get the current status of a share, enter:

```
# zfs get sharenfs mypool/usr/home
NAME             PROPERTY  VALUE    SOURCE
mypool/usr/home  sharenfs  on       local
# zfs get sharesmb mypool/usr/home
NAME             PROPERTY  VALUE    SOURCE
mypool/usr/home  sharesmb  off      local
```

To enable sharing of a dataset, enter:

```
#  zfs set sharenfs=on mypool/usr/home
```

Set other options for sharing datasets through NFS, such as `-alldirs`, `-maproot` and `-network`. To set options on a dataset shared through NFS, enter:

```
#  zfs set sharenfs="-alldirs,-maproot=root,-network=192.168.1.0/24" mypool/usr/home
```

22.4.5. Managing Snapshots

Snapshots are one of the most powerful features of ZFS. A snapshot provides a read-only, point-in-time copy of the dataset. With Copy-On-Write (COW), ZFS creates snapshots fast by preserving older versions of the data on disk. If no snapshots exist, ZFS reclaims space for future use when data is rewritten or deleted. Snapshots preserve disk space by recording just the differences between the current dataset and a previous version. Allowing snapshots on whole datasets, not on individual files or directories. A snapshot from a dataset duplicates everything contained in it. This includes the file system properties, files, directories, permissions, and so on. Snapshots use no extra space

when first created, but consume space as the blocks they reference change. Recursive snapshots taken with `-r` create snapshots with the same name on the dataset and its children, providing a consistent moment-in-time snapshot of the file systems. This can be important when an application has files on related datasets or that depend upon each other. Without snapshots, a backup would have copies of the files from different points in time.

Snapshots in ZFS provide a variety of features that even other file systems with snapshot functionality lack. A typical example of snapshot use is as a quick way of backing up the current state of the file system when performing a risky action like a software installation or a system upgrade. If the action fails, rolling back to the snapshot returns the system to the same state when creating the snapshot. If the upgrade was successful, delete the snapshot to free up space. Without snapshots, a failed upgrade often requires restoring backups, which is tedious, time consuming, and may require downtime during which the system is unusable. Rolling back to snapshots is fast, even while the system is running in normal operation, with little or no downtime. The time savings are enormous with multi-terabyte storage systems considering the time required to copy the data from backup. Snapshots are not a replacement for a complete backup of a pool, but offer a quick and easy way to store a dataset copy at a specific time.

22.4.5.1. Creating Snapshots

To create snapshots, use `zfs snapshot dataset@snapshotname`. Adding `-r` creates a snapshot recursively, with the same name on all child datasets.

Create a recursive snapshot of the entire pool:

```
# zfs list -t all
NAME                                  USED  AVAIL  REFER  MOUNTPOINT
mypool                                780M  93.2G   144K  none
mypool/ROOT                           777M  93.2G   144K  none
mypool/ROOT/default                   777M  93.2G   777M  /
mypool/tmp                            176K  93.2G   176K  /tmp
mypool/usr                            616K  93.2G   144K  /usr
mypool/usr/home                       184K  93.2G   184K  /usr/home
mypool/usr/ports                      144K  93.2G   144K  /usr/ports
mypool/usr/src                        144K  93.2G   144K  /usr/src
mypool/var                           1.29M  93.2G   616K  /var
mypool/var/crash                      148K  93.2G   148K  /var/crash
mypool/var/log                        178K  93.2G   178K  /var/log
mypool/var/mail                       144K  93.2G   144K  /var/mail
mypool/var/newname                   87.5K  93.2G  87.5K  /var/newname
mypool/var/newname@new_snapshot_name     0      -  87.5K  -
mypool/var/tmp                        152K  93.2G   152K  /var/tmp
# zfs snapshot -r mypool@my_recursive_snapshot
# zfs list -t snapshot
NAME                                      USED  AVAIL  REFER  MOUNTPOINT
mypool@my_recursive_snapshot                 0      -   144K  -
mypool/ROOT@my_recursive_snapshot            0      -   144K  -
mypool/ROOT/default@my_recursive_snapshot     0      -   777M  -
mypool/tmp@my_recursive_snapshot             0      -   176K  -
mypool/usr@my_recursive_snapshot             0      -   144K  -
```

```
mypool/usr/home@my_recursive_snapshot          0      -   184K  -
mypool/usr/ports@my_recursive_snapshot         0      -   144K  -
mypool/usr/src@my_recursive_snapshot           0      -   144K  -
mypool/var@my_recursive_snapshot               0      -   616K  -
mypool/var/crash@my_recursive_snapshot         0      -   148K  -
mypool/var/log@my_recursive_snapshot           0      -   178K  -
mypool/var/mail@my_recursive_snapshot          0      -   144K  -
mypool/var/newname@new_snapshot_name           0      -  87.5K  -
mypool/var/newname@my_recursive_snapshot       0      -  87.5K  -
mypool/var/tmp@my_recursive_snapshot           0      -   152K  -
```

Snapshots are not shown by a normal `zfs list` operation. To list snapshots, append `-t snapshot` to `zfs list`. `-t all` displays both file systems and snapshots.

Snapshots are not mounted directly, showing no path in the `MOUNTPOINT` column. ZFS does not mention available disk space in the `AVAIL` column, as snapshots are read-only after their creation. Compare the snapshot to the original dataset:

```
# zfs list -rt all mypool/usr/home
NAME                                 USED  AVAIL  REFER  MOUNTPOINT
mypool/usr/home                      184K  93.2G   184K  /usr/home
mypool/usr/home@my_recursive_snapshot    0      -   184K  -
```

Displaying both the dataset and the snapshot together reveals how snapshots work in COW fashion. They save the changes (*delta*) made and not the complete file system contents all over again. This means that snapshots take little space when making changes. Observe space usage even more by copying a file to the dataset, then creating a second snapshot:

```
# cp /etc/passwd /var/tmp
# zfs snapshot mypool/var/tmp@after_cp
# zfs list -rt all mypool/var/tmp
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp                      206K  93.2G   118K  /var/tmp
mypool/var/tmp@my_recursive_snapshot   88K      -   152K  -
mypool/var/tmp@after_cp                0      -   118K  -
```

The second snapshot contains the changes to the dataset after the copy operation. This yields enormous space savings. Notice that the size of the snapshot `mypool/var/tmp@my_recursive_snapshot` also changed in the `USED` column to show the changes between itself and the snapshot taken afterwards.

22.4.5.2. Comparing Snapshots

ZFS provides a built-in command to compare the differences in content between two snapshots. This is helpful with a lot of snapshots taken over time when the user wants to see how the file system has changed over time. For example, `zfs diff` lets a user find the latest snapshot that still contains a file deleted by accident. Doing this for the two snapshots created in the previous section

yields this output:

```
# zfs list -rt all mypool/var/tmp
NAME                                  USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp                        206K  93.2G   118K  /var/tmp
mypool/var/tmp@my_recursive_snapshot   88K      -   152K  -
mypool/var/tmp@after_cp                 0      -   118K  -
# zfs diff mypool/var/tmp@my_recursive_snapshot
M       /var/tmp/
+       /var/tmp/passwd
```

The command lists the changes between the specified snapshot (in this case `mypool/var/tmp@my_recursive_snapshot`) and the live file system. The first column shows the change type:

+	Adding the path or file.
-	Deleting the path or file.
M	Modifying the path or file.
R	Renaming the path or file.

Comparing the output with the table, it becomes clear that ZFS added passwd after creating the snapshot `mypool/var/tmp@my_recursive_snapshot`. This also resulted in a modification to the parent directory mounted at `/var/tmp`.

Comparing two snapshots is helpful when using the ZFS replication feature to transfer a dataset to a different host for backup purposes.

Compare two snapshots by providing the full dataset name and snapshot name of both datasets:

```
# cp /var/tmp/passwd /var/tmp/passwd.copy
# zfs snapshot mypool/var/tmp@diff_snapshot
# zfs diff mypool/var/tmp@my_recursive_snapshot mypool/var/tmp@diff_snapshot
M       /var/tmp/
+       /var/tmp/passwd
+       /var/tmp/passwd.copy
# zfs diff mypool/var/tmp@my_recursive_snapshot mypool/var/tmp@after_cp
M       /var/tmp/
+       /var/tmp/passwd
```

A backup administrator can compare two snapshots received from the sending host and determine the actual changes in the dataset. See the Replication section for more information.

22.4.5.3. Snapshot Rollback

When at least one snapshot is available, roll back to it at any time. Most often this is the case when the current state of the dataset is no longer valid or an older version is preferred. Scenarios such as local development tests gone wrong, botched system updates hampering the system functionality,

or the need to restore deleted files or directories are all too common occurrences. To roll back a snapshot, use `zfs rollback snapshotname`. If a lot of changes are present, the operation will take a long time. During that time, the dataset always remains in a consistent state, much like a database that conforms to ACID principles is performing a rollback. This is happening while the dataset is live and accessible without requiring a downtime. Once the snapshot rolled back, the dataset has the same state as it had when the snapshot was originally taken. Rolling back to a snapshot discards all other data in that dataset not part of the snapshot. Taking a snapshot of the current state of the dataset before rolling back to a previous one is a good idea when requiring some data later. This way, the user can roll back and forth between snapshots without losing data that is still valuable.

In the first example, roll back a snapshot because a careless `rm` operation removed more data than intended.

```
# zfs list -rt all mypool/var/tmp
NAME                                  USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp                        262K  93.2G   120K  /var/tmp
mypool/var/tmp@my_recursive_snapshot   88K      -   152K  -
mypool/var/tmp@after_cp              53.5K      -   118K  -
mypool/var/tmp@diff_snapshot             0      -   120K  -
# ls /var/tmp
passwd          passwd.copy     vi.recover
# rm /var/tmp/passwd*
# ls /var/tmp
vi.recover
```

At this point, the user notices the removal of extra files and wants them back. ZFS provides an easy way to get them back using rollbacks, when performing snapshots of important data on a regular basis. To get the files back and start over from the last snapshot, issue the command:

```
# zfs rollback mypool/var/tmp@diff_snapshot
# ls /var/tmp
passwd          passwd.copy     vi.recover
```

The rollback operation restored the dataset to the state of the last snapshot. Rolling back to a snapshot taken much earlier with other snapshots taken afterwards is also possible. When trying to do this, ZFS will issue this warning:

```
# zfs list -rt snapshot mypool/var/tmp
AME                                  USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp@my_recursive_snapshot   88K      -   152K  -
mypool/var/tmp@after_cp              53.5K      -   118K  -
mypool/var/tmp@diff_snapshot             0      -   120K  -
# zfs rollback mypool/var/tmp@my_recursive_snapshot
cannot rollback to 'mypool/var/tmp@my_recursive_snapshot': more recent snapshots exist
use '-r' to force deletion of the following snapshots:
mypool/var/tmp@after_cp
```

```
mypool/var/tmp@diff_snapshot
```

This warning means that snapshots exist between the current state of the dataset and the snapshot to which the user wants to roll back. To complete the rollback delete these snapshots. ZFS cannot track all the changes between different states of the dataset, because snapshots are read-only. ZFS will not delete the affected snapshots unless the user specifies `-r` to confirm that this is the desired action. If that is the intention, and understanding the consequences of losing all intermediate snapshots, issue the command:

```
# zfs rollback -r mypool/var/tmp@my_recursive_snapshot
# zfs list -rt snapshot mypool/var/tmp
NAME                                     USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp@my_recursive_snapshot     8K      -   152K  -
# ls /var/tmp
vi.recover
```

The output from `zfs list -t snapshot` confirms the removal of the intermediate snapshots as a result of `zfs rollback -r`.

22.4.5.4. Restoring Individual Files from Snapshots

Snapshots live in a hidden directory under the parent dataset: .zfs/snapshots/snapshotname. By default, these directories will not show even when executing a standard `ls -a` . Although the directory doesn't show, access it like any normal directory. The property named `snapdir` controls whether these hidden directories show up in a directory listing. Setting the property to `visible` allows them to appear in the output of `ls` and other commands that deal with directory contents.

```
# zfs get snapdir mypool/var/tmp
NAME            PROPERTY  VALUE    SOURCE
mypool/var/tmp  snapdir   hidden   default
# ls -a /var/tmp
.               ..              passwd          vi.recover
# zfs set snapdir=visible mypool/var/tmp
# ls -a /var/tmp
.               ..              .zfs            passwd          vi.recover
```

Restore individual files to a previous state by copying them from the snapshot back to the parent dataset. The directory structure below .zfs/snapshot has a directory named like the snapshots taken earlier to make it easier to identify them. The next example shows how to restore a file from the hidden .zfs directory by copying it from the snapshot containing the latest version of the file:

```
# rm /var/tmp/passwd
# ls -a /var/tmp
.               ..              .zfs            vi.recover
# ls /var/tmp/.zfs/snapshot
after_cp                my_recursive_snapshot
# ls /var/tmp/.zfs/snapshot/after_cp
```

```
passwd          vi.recover
# cp /var/tmp/.zfs/snapshot/after_cp/passwd /var/tmp
```

Even if the `snapdir` property is set to hidden, running `ls .zfs/snapshot` will still list the contents of that directory. The administrator decides whether to display these directories. This is a per-dataset setting. Copying files or directories from this hidden .zfs/snapshot is simple enough. Trying it the other way around results in this error:

```
# cp /etc/rc.conf /var/tmp/.zfs/snapshot/after_cp/
cp: /var/tmp/.zfs/snapshot/after_cp/rc.conf: Read-only file system
```

The error reminds the user that snapshots are read-only and cannot change after creation. Copying files into and removing them from snapshot directories are both disallowed because that would change the state of the dataset they represent.

Snapshots consume space based on how much the parent file system has changed since the time of the snapshot. The `written` property of a snapshot tracks the space the snapshot uses.

To destroy snapshots and reclaim the space, use `zfs destroy dataset@snapshot`. Adding `-r` recursively removes all snapshots with the same name under the parent dataset. Adding `-n -v` to the command displays a list of the snapshots to be deleted and an estimate of the space it would reclaim without performing the actual destroy operation.

22.4.6. Managing Clones

A clone is a copy of a snapshot treated more like a regular dataset. Unlike a snapshot, a clone is writeable and mountable, and has its own properties. After creating a clone using `zfs clone`, destroying the originating snapshot is impossible. To reverse the child/parent relationship between the clone and the snapshot use `zfs promote`. Promoting a clone makes the snapshot become a child of the clone, rather than of the original parent dataset. This will change how ZFS accounts for the space, but not actually change the amount of space consumed. Mounting the clone anywhere within the ZFS file system hierarchy is possible, not only below the original location of the snapshot.

To show the clone feature use this example dataset:

```
# zfs list -rt all camino/home/joe
NAME                    USED  AVAIL  REFER  MOUNTPOINT
camino/home/joe         108K   1.3G    87K  /usr/home/joe
camino/home/joe@plans    21K      -  85.5K  -
camino/home/joe@backup    0K      -    87K  -
```

A typical use for clones is to experiment with a specific dataset while keeping the snapshot around to fall back to in case something goes wrong. Since snapshots cannot change, create a read/write clone of a snapshot. After achieving the desired result in the clone, promote the clone to a dataset and remove the old file system. Removing the parent dataset is not strictly necessary, as the clone and dataset can coexist without problems.

```
# zfs clone camino/home/joe@backup camino/home/joenew
# ls /usr/home/joe*
/usr/home/joe:
backup.txz     plans.txt

/usr/home/joenew:
backup.txz     plans.txt
# df -h /usr/home
Filesystem          Size    Used   Avail Capacity  Mounted on
usr/home/joe        1.3G     31k    1.3G     0%    /usr/home/joe
usr/home/joenew     1.3G     31k    1.3G     0%    /usr/home/joenew
```

Creating a clone makes it an exact copy of the state the dataset was in when taking the snapshot. Changing the clone independently from its originating dataset is possible now. The connection between the two is the snapshot. ZFS records this connection in the property `origin`. Promoting the clone with `zfs promote` makes the clone an independent dataset. This removes the value of the `origin` property and disconnects the newly independent dataset from the snapshot. This example shows it:

```
# zfs get origin camino/home/joenew
NAME                 PROPERTY  VALUE                     SOURCE
camino/home/joenew   origin    camino/home/joe@backup    -
# zfs promote camino/home/joenew
# zfs get origin camino/home/joenew
NAME                 PROPERTY  VALUE   SOURCE
camino/home/joenew   origin    -       -
```

After making some changes like copying loader.conf to the promoted clone, for example, the old directory becomes obsolete in this case. Instead, the promoted clone can replace it. To do this, `zfs destroy` the old dataset first and then `zfs rename` the clone to the old dataset name (or to an entirely different name).

```
# cp /boot/defaults/loader.conf /usr/home/joenew
# zfs destroy -f camino/home/joe
# zfs rename camino/home/joenew camino/home/joe
# ls /usr/home/joe
backup.txz     loader.conf     plans.txt
# df -h /usr/home
Filesystem          Size    Used   Avail Capacity  Mounted on
usr/home/joe        1.3G    128k    1.3G     0%    /usr/home/joe
```

The cloned snapshot is now an ordinary dataset. It contains all the data from the original snapshot plus the files added to it like loader.conf. Clones provide useful features to ZFS users in different scenarios. For example, provide jails as snapshots containing different sets of installed applications. Users can clone these snapshots and add their own applications as they see fit. Once satisfied with the changes, promote the clones to full datasets and provide them to end users to work with like

they would with a real dataset. This saves time and administrative overhead when providing these jails.

22.4.7. Replication

Keeping data on a single pool in one location exposes it to risks like theft and natural or human disasters. Making regular backups of the entire pool is vital. ZFS provides a built-in serialization feature that can send a stream representation of the data to standard output. Using this feature, storing this data on another pool connected to the local system is possible, as is sending it over a network to another system. Snapshots are the basis for this replication (see the section on ZFS snapshots). The commands used for replicating data are `zfs send` and `zfs receive`.

These examples show ZFS replication with these two pools:

```
# zpool list
NAME    SIZE  ALLOC   FREE   CKPOINT  EXPANDSZ   FRAG   CAP  DEDUP  HEALTH  ALTROOT
backup  960M    77K   896M         -         -     0%    0%  1.00x  ONLINE  -
mypool  984M  43.7M   940M         -         -     0%    4%  1.00x  ONLINE  -
```

The pool named *mypool* is the primary pool where writing and reading data happens on a regular basis. Using a second standby pool *backup* in case the primary pool becomes unavailable. Note that this fail-over is not done automatically by ZFS, but must be manually done by a system administrator when needed. Use a snapshot to provide a consistent file system version to replicate. After creating a snapshot of *mypool*, copy it to the *backup* pool by replicating snapshots. This does not include changes made since the most recent snapshot.

```
# zfs snapshot mypool@backup1
# zfs list -t snapshot
NAME                    USED  AVAIL  REFER  MOUNTPOINT
mypool@backup1             0      -  43.6M  -
```

Now that a snapshot exists, use `zfs send` to create a stream representing the contents of the snapshot. Store this stream as a file or receive it on another pool. Write the stream to standard output, but redirect to a file or pipe or an error appears:

```
# zfs send mypool@backup1
Error: Stream can not be written to a terminal.
You must redirect standard output.
```

To back up a dataset with `zfs send`, redirect to a file located on the mounted backup pool. Ensure that the pool has enough free space to accommodate the size of the sent snapshot, which means the data contained in the snapshot, not the changes from the previous snapshot.

```
# zfs send mypool@backup1 > /backup/backup1
# zpool list
NAME    SIZE  ALLOC   FREE   CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP  HEALTH  ALTROOT
```

```
backup  960M  63.7M   896M        -         -     0%     6%  1.00x  ONLINE  -
mypool  984M  43.7M   940M        -         -     0%     4%  1.00x  ONLINE  -
```

The `zfs send` transferred all the data in the snapshot called *backup1* to the pool named *backup*. To create and send these snapshots automatically, use a cron(8) job.

Instead of storing the backups as archive files, ZFS can receive them as a live file system, allowing direct access to the backed up data. To get to the actual data contained in those streams, use `zfs receive` to transform the streams back into files and directories. The example below combines `zfs send` and `zfs receive` using a pipe to copy the data from one pool to another. Use the data directly on the receiving pool after the transfer is complete. It is only possible to replicate a dataset to an empty dataset.

```
# zfs snapshot mypool@replica1
# zfs send -v mypool@replica1 | zfs receive backup/mypool
send from @ to mypool@replica1 estimated size is 50.1M
total estimated size is 50.1M
TIME        SENT   SNAPSHOT

# zpool list
NAME    SIZE  ALLOC   FREE   CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP  HEALTH  ALTROOT
backup  960M  63.7M   896M        -         -     0%     6%  1.00x  ONLINE  -
mypool  984M  43.7M   940M        -         -     0%     4%  1.00x  ONLINE  -
```

22.4.7.1. Incremental Backups

`zfs send` can also determine the difference between two snapshots and send individual differences between the two. This saves disk space and transfer time. For example:

```
# zfs snapshot mypool@replica2
# zfs list -t snapshot
NAME                    USED  AVAIL  REFER  MOUNTPOINT
mypool@replica1         5.72M      -  43.6M  -
mypool@replica2             0      -  44.1M  -
# zpool list
NAME    SIZE  ALLOC   FREE   CKPOINT  EXPANDSZ   FRAG   CAP  DEDUP  HEALTH  ALTROOT
backup  960M  61.7M   898M        -         -     0%    6%  1.00x  ONLINE  -
mypool  960M  50.2M   910M        -         -     0%    5%  1.00x  ONLINE  -
```

Create a second snapshot called *replica2*. This second snapshot contains changes made to the file system between now and the previous snapshot, *replica1*. Using `zfs send -i` and indicating the pair of snapshots generates an incremental replica stream containing the changed data. This succeeds if the initial snapshot already exists on the receiving side.

```
# zfs send -v -i mypool@replica1 mypool@replica2 | zfs receive /backup/mypool
send from @replica1 to mypool@replica2 estimated size is 5.02M
total estimated size is 5.02M
```

```
TIME        SENT   SNAPSHOT

# zpool list
NAME    SIZE  ALLOC   FREE   CKPOINT  EXPANDSZ   FRAG  CAP  DEDUP  HEALTH  ALTROOT
backup  960M  80.8M   879M         -         -     0%   8%  1.00x  ONLINE  -
mypool  960M  50.2M   910M         -         -     0%   5%  1.00x  ONLINE  -

# zfs list
NAME                          USED  AVAIL  REFER  MOUNTPOINT
backup                       55.4M   240G   152K  /backup
backup/mypool                55.3M   240G  55.2M  /backup/mypool
mypool                       55.6M  11.6G  55.0M  /mypool

# zfs list -t snapshot
NAME                                      USED  AVAIL  REFER  MOUNTPOINT
backup/mypool@replica1                    104K      -  50.2M  -
backup/mypool@replica2                       0      -  55.2M  -
mypool@replica1                           29.9K      -  50.0M  -
mypool@replica2                              0      -  55.0M  -
```

The incremental stream replicated the changed data rather than the entirety of *replica1*. Sending the differences alone took much less time to transfer and saved disk space by not copying the whole pool each time. This is useful when replicating over a slow network or one charging per transferred byte.

A new file system, *backup/mypool*, is available with the files and data from the pool *mypool*. Specifying `-p` copies the dataset properties including compression settings, quotas, and mount points. Specifying `-R` copies all child datasets of the dataset along with their properties. Automate sending and receiving to create regular backups on the second pool.

22.4.7.2. Sending Encrypted Backups over SSH

Sending streams over the network is a good way to keep a remote backup, but it does come with a drawback. Data sent over the network link is not encrypted, allowing anyone to intercept and transform the streams back into data without the knowledge of the sending user. This is undesirable when sending the streams over the internet to a remote host. Use SSH to securely encrypt data sent over a network connection. Since ZFS requires redirecting the stream from standard output, piping it through SSH is easy. To keep the contents of the file system encrypted in transit and on the remote system, consider using PEFS.

Change some settings and take security precautions first. This describes the necessary steps required for the `zfs send` operation; for more information on SSH, see OpenSSH.

Change the configuration as follows:

- Passwordless SSH access between sending and receiving host using SSH keys
- ZFS requires the privileges of the `root` user to send and receive streams. This requires logging in to the receiving system as `root`.
- Security reasons prevent `root` from logging in by default.

- Use the ZFS Delegation system to allow a non-`root` user on each system to perform the respective send and receive operations. On the sending system:

```
# zfs allow -u someuser send,snapshot mypool
```

- To mount the pool, the unprivileged user must own the directory, and regular users need permission to mount file systems.

On the receiving system:

```
# sysctl vfs.usermount=1
vfs.usermount: 0 -> 1
# echo vfs.usermount=1 >> /etc/sysctl.conf
# zfs create recvpool/backup
# zfs allow -u someuser create,mount,receive recvpool/backup
# chown someuser /recvpool/backup
```

The unprivileged user can receive and mount datasets now, and replicates the *home* dataset to the remote system:

```
% zfs snapshot -r mypool/home@monday
% zfs send -R mypool/home@monday | ssh someuser@backuphost zfs recv -dvu
recvpool/backup
```

Create a recursive snapshot called *monday* of the file system dataset *home* on the pool *mypool*. Then `zfs send -R` includes the dataset, all child datasets, snapshots, clones, and settings in the stream. Pipe the output through SSH to the waiting `zfs receive` on the remote host *backuphost*. Using an IP address or fully qualified domain name is good practice. The receiving machine writes the data to the *backup* dataset on the *recvpool* pool. Adding `-d` to `zfs recv` overwrites the name of the pool on the receiving side with the name of the snapshot. `-u` causes the file systems to not mount on the receiving side. Using `-v` shows more details about the transfer, including the elapsed time and the amount of data transferred.

22.4.8. Dataset, User, and Group Quotas

Use Dataset quotas to restrict the amount of space consumed by a particular dataset. Reference Quotas work in much the same way, but count the space used by the dataset itself, excluding snapshots and child datasets. Similarly, use user and group quotas to prevent users or groups from using up all the space in the pool or dataset.

The following examples assume that the users already exist in the system. Before adding a user to the system, make sure to create their home dataset first and set the `mountpoint` to `/home/bob`. Then, create the user and make the home directory point to the dataset's `mountpoint` location. This will properly set owner and group permissions without shadowing any pre-existing home directory paths that might exist.

To enforce a dataset quota of 10 GB for storage/home/bob:

```
# zfs set quota=10G storage/home/bob
```

To enforce a reference quota of 10 GB for storage/home/bob:

```
# zfs set refquota=10G storage/home/bob
```

To remove a quota of 10 GB for storage/home/bob:

```
# zfs set quota=none storage/home/bob
```

The general format is `userquota@user=size`, and the user's name must be in one of these formats:

- POSIX compatible name such as *joe*.
- POSIX numeric ID such as *789*.
- SID name such as *joe.bloggs@example.com*.
- SID numeric ID such as *S-1-123-456-789*.

For example, to enforce a user quota of 50 GB for the user named *joe*:

```
# zfs set userquota@joe=50G
```

To remove any quota:

```
# zfs set userquota@joe=none
```



User quota properties are not displayed by `zfs get all`. Non-`root` users can't see other's quotas unless granted the `userquota` privilege. Users with this privilege are able to view and set everyone's quota.

The general format for setting a group quota is: `groupquota@group=size`.

To set the quota for the group *firstgroup* to 50 GB, use:

```
# zfs set groupquota@firstgroup=50G
```

To remove the quota for the group *firstgroup*, or to make sure that one is not set, instead use:

```
# zfs set groupquota@firstgroup=none
```

As with the user quota property, non-`root` users can see the quotas associated with the groups to which they belong. A user with the `groupquota` privilege or `root` can view and set all quotas for all groups.

To display the amount of space used by each user on a file system or snapshot along with any quotas, use `zfs userspace`. For group information, use `zfs groupspace`. For more information about supported options or how to display specific options alone, refer to zfs(1).

Privileged users and `root` can list the quota for storage/home/bob using:

```
# zfs get quota storage/home/bob
```

22.4.9. Reservations

Reservations guarantee an always-available amount of space on a dataset. The reserved space will not be available to any other dataset. This useful feature ensures that free space is available for an important dataset or log files.

The general format of the `reservation` property is `reservation=size`, so to set a reservation of 10 GB on storage/home/bob, use:

```
# zfs set reservation=10G storage/home/bob
```

To clear any reservation:

```
# zfs set reservation=none storage/home/bob
```

The same principle applies to the `refreservation` property for setting a Reference Reservation, with the general format `refreservation=size`.

This command shows any reservations or refreservations that exist on storage/home/bob:

```
# zfs get reservation storage/home/bob
# zfs get refreservation storage/home/bob
```

22.4.10. Compression

ZFS provides transparent compression. Compressing data written at the block level saves space and also increases disk throughput. If data compresses by 25% the compressed data writes to the disk at the same rate as the uncompressed version, resulting in an effective write speed of 125%. Compression can also be a great alternative to Deduplication because it does not require extra memory.

ZFS offers different compression algorithms, each with different trade-offs. The introduction of LZ4 compression in ZFS v5000 enables compressing the entire pool without the large performance

trade-off of other algorithms. The biggest advantage to LZ4 is the *early abort* feature. If LZ4 does not achieve at least 12.5% compression in the header part of the data, ZFS writes the block uncompressed to avoid wasting CPU cycles trying to compress data that is either already compressed or uncompressible. For details about the different compression algorithms available in ZFS, see the Compression entry in the terminology section.

The administrator can see the effectiveness of compression using dataset properties.

```
# zfs get used,compressratio,compression,logicalused mypool/compressed_dataset
NAME        PROPERTY          VALUE     SOURCE
mypool/compressed_dataset  used              449G      -
mypool/compressed_dataset  compressratio     1.11x     -
mypool/compressed_dataset  compression       lz4       local
mypool/compressed_dataset  logicalused       496G      -
```

The dataset is using 449 GB of space (the used property). Without compression, it would have taken 496 GB of space (the `logicalused` property). This results in a 1.11:1 compression ratio.

Compression can have an unexpected side effect when combined with User Quotas. User quotas restrict how much actual space a user consumes on a dataset *after compression*. If a user has a quota of 10 GB, and writes 10 GB of compressible data, they will still be able to store more data. If they later update a file, say a database, with more or less compressible data, the amount of space available to them will change. This can result in the odd situation where a user did not increase the actual amount of data (the `logicalused` property), but the change in compression caused them to reach their quota limit.

Compression can have a similar unexpected interaction with backups. Quotas are often used to limit data storage to ensure there is enough backup space available. Since quotas do not consider compression ZFS may write more data than would fit with uncompressed backups.

22.4.11. Zstandard Compression

OpenZFS 2.0 added a new compression algorithm. Zstandard (Zstd) offers higher compression ratios than the default LZ4 while offering much greater speeds than the alternative, gzip. OpenZFS 2.0 is available starting with FreeBSD 12.1-RELEASE via sysutils/openzfs and has been the default since FreeBSD 13.0-RELEASE.

Zstd provides a large selection of compression levels, providing fine-grained control over performance versus compression ratio. One of the main advantages of Zstd is that the decompression speed is independent of the compression level. For data written once but read often, Zstd allows the use of the highest compression levels without a read performance penalty.

Even with frequent data updates, enabling compression often provides higher performance. One of the biggest advantages comes from the compressed ARC feature. ZFS's Adaptive Replacement Cache (ARC) caches the compressed version of the data in RAM, decompressing it each time. This allows the same amount of RAM to store more data and metadata, increasing the cache hit ratio.

ZFS offers 19 levels of Zstd compression, each offering incrementally more space savings in exchange for slower compression. The default level is `zstd-3` and offers greater compression than

LZ4 without being much slower. Levels above 10 require large amounts of memory to compress each block and systems with less than 16 GB of RAM should not use them. ZFS uses a selection of the Zstd_fast_ levels also, which get correspondingly faster but supports lower compression ratios. ZFS supports `zstd-fast-1` through `zstd-fast-10`, `zstd-fast-20` through `zstd-fast-100` in increments of 10, and `zstd-fast-500` and `zstd-fast-1000` which provide minimal compression, but offer high performance.

If ZFS is not able to get the required memory to compress a block with Zstd, it will fall back to storing the block uncompressed. This is unlikely to happen except at the highest levels of Zstd on memory constrained systems. ZFS counts how often this has occurred since loading the ZFS module with `kstat.zfs.misc.zstd.compress_alloc_fail`.

22.4.12. Deduplication

When enabled, deduplication uses the checksum of each block to detect duplicate blocks. When a new block is a duplicate of an existing block, ZFS writes a new reference to the existing data instead of the whole duplicate block. Tremendous space savings are possible if the data contains a lot of duplicated files or repeated information. Warning: deduplication requires a large amount of memory, and enabling compression instead provides most of the space savings without the extra cost.

To activate deduplication, set the `dedup` property on the target pool:

```
# zfs set dedup=on pool
```

Deduplicating only affects new data written to the pool. Merely activating this option will not deduplicate data already written to the pool. A pool with a freshly activated deduplication property will look like this example:

```
# zpool list
NAME  SIZE ALLOC  FREE   CKPOINT  EXPANDSZ   FRAG   CAP   DEDUP   HEALTH   ALTROOT
pool 2.84G 2.19M 2.83G         -         -     0%    0%   1.00x   ONLINE   -
```

The `DEDUP` column shows the actual rate of deduplication for the pool. A value of `1.00x` shows that data has not deduplicated yet. The next example copies some system binaries three times into different directories on the deduplicated pool created above.

```
# for d in dir1 dir2 dir3; do
> mkdir $d && cp -R /usr/bin $d &
> done
```

To observe deduplicating of redundant data, use:

```
# zpool list
NAME SIZE  ALLOC  FREE   CKPOINT  EXPANDSZ   FRAG  CAP   DEDUP   HEALTH   ALTROOT
```

```
pool 2.84G 20.9M 2.82G        -         -     0%   0%   3.00x   ONLINE   -
```

The `DEDUP` column shows a factor of `3.00x`. Detecting and deduplicating copies of the data uses a third of the space. The potential for space savings can be enormous, but comes at the cost of having enough memory to keep track of the deduplicated blocks.

Deduplication is not always beneficial when the data in a pool is not redundant. ZFS can show potential space savings by simulating deduplication on an existing pool:

```
# zdb -S pool
Simulated DDT histogram:

bucket              allocated                       referenced
______   ______________________________   ______________________________
refcnt   blocks   LSIZE   PSIZE   DSIZE   blocks   LSIZE   PSIZE   DSIZE
------   ------   -----   -----   -----   ------   -----   -----   -----
     1    2.58M    289G    264G    264G    2.58M    289G    264G    264G
     2     206K   12.6G   10.4G   10.4G     430K   26.4G   21.6G   21.6G
     4    37.6K    692M    276M    276M     170K   3.04G   1.26G   1.26G
     8    2.18K   45.2M   19.4M   19.4M    20.0K    425M    176M    176M
    16      174   2.83M   1.20M   1.20M    3.33K   48.4M   20.4M   20.4M
    32       40   2.17M    222K    222K    1.70K   97.2M   9.91M   9.91M
    64        9     56K   10.5K   10.5K      865   4.96M    948K    948K
   128        2   9.50K      2K      2K      419   2.11M    438K    438K
   256        5   61.5K     12K     12K    1.90K   23.0M   4.47M   4.47M
    1K        2      1K      1K      1K    2.98K   1.49M   1.49M   1.49M
 Total    2.82M    303G    275G    275G    3.20M    319G    287G    287G

dedup = 1.05, compress = 1.11, copies = 1.00, dedup * compress / copies = 1.16
```

After `zdb -S` finishes analyzing the pool, it shows the space reduction ratio that activating deduplication would achieve. In this case, `1.16` is a poor space saving ratio mainly provided by compression. Activating deduplication on this pool would not save any amount of space, and is not worth the amount of memory required to enable deduplication. Using the formula *ratio* = *dedup* * *compress* / *copies*, system administrators can plan the storage allocation, deciding whether the workload will contain enough duplicate blocks to justify the memory requirements. If the data is reasonably compressible, the space savings may be good. Good practice is to enable compression first as compression also provides greatly increased performance. Enable deduplication in cases where savings are considerable and with enough available memory for the DDT.

22.4.13. ZFS and Jails

Use `zfs jail` and the corresponding `jailed` property to delegate a ZFS dataset to a Jail. `zfs jail jailid` attaches a dataset to the specified jail, and `zfs unjail` detaches it. To control the dataset from within a jail, set the `jailed` property. ZFS forbids mounting a jailed dataset on the host because it may have mount points that would compromise the security of the host.

22.5. Delegated Administration

A comprehensive permission delegation system allows unprivileged users to perform ZFS administration functions. For example, if each user's home directory is a dataset, users need permission to create and destroy snapshots of their home directories. A user performing backups can get permission to use replication features. ZFS allows a usage statistics script to run with access to only the space usage data for all users. Delegating the ability to delegate permissions is also possible. Permission delegation is possible for each subcommand and most properties.

22.5.1. Delegating Dataset Creation

`zfs allow someuser create mydataset` gives the specified user permission to create child datasets under the selected parent dataset. A caveat: creating a new dataset involves mounting it. That requires setting the FreeBSD `vfs.usermount` sysctl(8) to `1` to allow non-root users to mount a file system. Another restriction aimed at preventing abuse: non-`root` users must own the mountpoint where mounting the file system.

22.5.2. Delegating Permission Delegation

`zfs allow someuser allow mydataset` gives the specified user the ability to assign any permission they have on the target dataset, or its children, to other users. If a user has the `snapshot` permission and the `allow` permission, that user can then grant the `snapshot` permission to other users.

22.6. Advanced Topics

22.6.1. Tuning

Adjust tunables to make ZFS perform best for different workloads.

- `vfs.zfs.arc.max` starting with 13.x (`vfs.zfs.arc_max` for 12.x) - Upper size of the ARC. The default is all RAM but 1 GB, or 5/8 of all RAM, whichever is more. Use a lower value if the system runs any other daemons or processes that may require memory. Adjust this value at runtime with sysctl(8) and set it in /boot/loader.conf or /etc/sysctl.conf.
- `vfs.zfs.arc.meta_limit` starting with 13.x (`vfs.zfs.arc_meta_limit` for 12.x) - Limit the amount of the ARC used to store metadata. The default is one fourth of `vfs.zfs.arc.max`. Increasing this value will improve performance if the workload involves operations on a large number of files and directories, or frequent metadata operations, at the cost of less file data fitting in the ARC. Adjust this value at runtime with sysctl(8) in /boot/loader.conf or /etc/sysctl.conf.
- `vfs.zfs.arc.min` starting with 13.x (`vfs.zfs.arc_min` for 12.x) - Lower size of the ARC. The default is one half of `vfs.zfs.arc.meta_limit`. Adjust this value to prevent other applications from pressuring out the entire ARC. Adjust this value at runtime with sysctl(8) and in /boot/loader.conf or /etc/sysctl.conf.
- `vfs.zfs.vdev.cache.size` - A preallocated amount of memory reserved as a cache for each device in the pool. The total amount of memory used will be this value multiplied by the number of devices. Set this value at boot time and in /boot/loader.conf.
- `vfs.zfs.min_auto_ashift` - Lower `ashift` (sector size) used automatically at pool creation time.